WASP Cloud Computing and Software Engineering - Module 2 Assignment

Aban Ansari-Önnestam

1 Introduction

My research is in the field of mathematical optimization and focuses on the development and analysis of algorithms for solving unconstrained nonlinear optimization problems. In particular, the research concerns Newton and quasi-Newton methods, which utilize exact or approximate second-order information to construct search directions. These methods aim to achieve faster and more reliable convergence than approaches based solely on first-order information. The research combines theoretical analysis with algorithmic development. A central objective is to understand the mathematical mechanisms that govern the behavior of optimization algorithms, including their convergence properties and the relationships between different algorithmic frameworks.

2 Lecture Principles

Verification vs. Validation In my research, I am primarily concerned with verification. The core of my research is ensuring that the algorithms we develop are theoretically sound and satisfy specific properties. We use formal mathematical proofs to ensure convergence at a defined rate, assuming that the problems satisfy certain criteria. This approach has drawbacks for practical implementation because interesting problems do not always match the assumptions needed for theoretical guarantees, such as convexity or smoothness. Additionally, finite precision in computing can affect results, especially when Hessians have large condition numbers or gradients are poorly scaled. To address this, we evaluate algorithms on large collections of benchmark problems and generate performance profiles. This covers cases where the problems do not match our assumptions and indicates how computer precision may affect the results. Additionally, validation is vital when comparing algorithms with identical theoretical guarantees because it can show that one outperforms another in practice.

Science vs. Engineering My research is more science-focused than engineering-focused, but important elements of both are present. As mentioned in the lecture, science focuses more on the question *"Why does this work?"*, whereas engineering covers *"How can we make it work reliably?"*. On the science side, we isolate the

mechanisms that lead to certain results. While it is known that methods using conjugate search directions or quasi-Newton Hessian approximations yield finite termination for strictly convex quadratic functions, part of my research focuses on understanding exactly why this occurs. We investigate which assumptions yield the results known to hold theoretically. Conversely, on the engineering side, although it is not the main focus, we must keep in mind that algorithms need to work reliably. Therefore, we cannot assume that operations such as finding the inverse of arbitrary dense matrices are easy. Additionally, for large-scale optimization problems, keeping multiple vectors in memory is not an option for a fast, reliable algorithm.

3 Guest Lecture Principles

Both topics discussed are from the guest lecture by Julian Frattini.

Problem-Space vs. Solution-Space Regarding the problem space, which focuses on the questions *"Why?"* and *"What?"*, and the solution space, which concerns *"How?"*, my research focuses mostly on the solution space. Building algorithms is mostly concerned with how to solve a given problem. Once the problem has been specified, the focus is on finding a computationally efficient iterative method that produces a solution. The impact of the problem space is usually not directly part of our research, but it can be seen in the types of problems for which we try to build algorithms. The problem space not only specifies which types of functions we should consider but also adds practical constraints to the algorithm development process. Although the availability of computational resources is not a variable we can control in our research, it is something we must keep in mind when doing our work.

Levels of Abstraction If we decompose the levels of abstraction into three, with the highest being the *context layer*, the middle the *requirements layer*, and the lowest the *system layer*, my research generally fits within the middle and lowest layers. The highest layer of context serves as the motivation for the research. Ensuring that the methods we develop solve problems efficiently and reliably provides the broader context for our research. The middle requirements layer contains the specific constraints placed on our research. These are factors we must keep in mind but have little freedom to change. The lowest level of abstraction is the most relevant to my work. When developing methods to solve problems, we focus on answering how to do something.

4 Data scientists, software engineers, and AI engineers

The discussion in this section is specifically based on the contents of Chapters 1 and 2 of the book *Machine Learning in Production* [Käs25].

Data Scientists vs. Software Engineers The book draws a clear distinction between the roles and backgrounds of data scientists and software engineers. For data scientists, the assumption is that they focus more on data, model development, hard metrics, and performance, with less focus on practical concerns. The book states that a larger proportion of data scientists have academic backgrounds with a more theoretical focus, and many have PhDs in statistics or machine learning. Conversely, it states that software engineers are primarily concerned with delivering usable products. Specifically, software engineers must keep in mind factors such as cost, user requirements, and maintainability. The conclusion is that both are needed to develop machine-learning products. I believe that the distinction is too strong and that the line between the roles of data scientists and software engineers is quite blurry. Even when their focus is on one specific area, data scientists must generally be aware of issues such as usability and scalability. Similarly, software engineers rely on knowledge of models and must use established theoretical metrics to choose methods.

System-Wide Views In general, my research on optimization methods is conducted in isolation. However, whether algorithms are successful is decided from a system-wide perspective. Factors such as scalability and reliability are integral to determining whether an algorithm is usable. Additionally, energy use, including the effects of higher memory or computational requirements, is considered very important, even though we judge the quality of the methods we develop using theoretical convergence guarantees.

AI Engineering Regarding how the distinction between data scientists and software engineers, as well as system-wide views, fits into AI engineering, it is clear that data scientists help design infrastructure but must adopt a system-wide view so that implementation by software engineers is feasible. With tools such as AI coding assistants, the lines between distinct roles are becoming blurred because these tools can provide support across different skills.

5 Paper Analysis

5.1 A Combinatorial Approach to Hyperparameter Optimization [Kha+24]

Core ideas and their software engineering importance. The paper looks at hyperparameter optimization from a testing perspective. Instead of trying all possible hyperparameter combinations using strategies such as grid search, it uses a combinatorial approach to systematically narrow the search space by exploring important interactions between parameters while keeping the number of experiments manageable. In practice, training models often involves many rounds of experimentation, which can become expensive and time-consuming. The paper borrows ideas from software testing to provide a more structured way to explore the search space. This can make training more efficient.

Relation to my research. My research concerns optimization algorithms for continuous, unconstrained nonlinear problems. Therefore, on the surface, this paper and my research do not have much in common because hyperparameter optimization generally focuses on mixed-integer nonlinear problems. However, my research may relate to this work through its application to model training. The algorithms I develop could be used in model training, whereas the hyperparameter optimization proposed in the paper would be used to choose inputs for those algorithms. The paper focuses on practical engineering concerns, such as reducing experimentation costs, while my research focuses on understanding why optimization algorithms work. Even so, both areas share the goal of finding better solutions with less computational effort.

Integration into a larger AI-intensive project. The paper tests its hyperparameter optimization scheme in classification tasks such as breast cancer classification. For such a project, large amounts of data would be required because of the need for accuracy. Additionally, architecture choices would be vital for ensuring accuracy. Therefore, training any test model would be expensive, and many models would need to be trained if an approach such as grid search were used to find appropriate hyperparameters. Using a combinatorial approach to hyperparameter optimization would reduce computational costs while supporting an accurate resulting model. My own research could contribute at a different level. While the paper helps determine which hyperparameter settings to use, optimization algorithms are still needed to train the models themselves. Better optimization methods could reduce training time and computational costs, making the overall system more efficient.

Adaptation of my research. One takeaway from the paper is that optimization does not happen in isolation. It is part of a larger engineering workflow involving experimentation, validation, and configuration choices. When I conduct performance

testing for validation, it is important to keep in mind inputs to my algorithms, such as step sizes or constants used in line searches.

5.2 Optimizing the Optimizer: An Example Showing the Power of LLM Code Generation [SB25]

Core ideas and their software engineering importance. The paper explores how generative AI can help improve existing optimization algorithms, going beyond the use of LLMs simply to generate code. Because machine-learning models are good at pattern recognition, the paper explores how they may help improve heuristic algorithms. In my research field, algorithms are generally developed by researchers, and the role of LLMs is to generate code for testing. This paper uses machine learning to generate the algorithms themselves, which achieve good results in testing.

Relation to my research. This paper focuses on developing heuristics using LLMs. In general, heuristics do not guarantee an optimal solution but instead provide a feasible solution that is "good." In contrast, my research concerns algorithms with strong theoretical guarantees. However, there is an opportunity to investigate whether LLMs can be applied to algorithms with strong theoretical guarantees to improve their formulation, increase computational efficiency, or suggest other improvements.

Integration into a larger AI-intensive project. For applications such as snow removal or airline routing, which generally rely on problem-specific heuristics, LLMs could be integrated into the design of those heuristics. Regarding my own research, we might study the theoretical guarantees of these AI-generated heuristics for issues such as safety, which is vital in problems such as airline routing.

Adaptation of my research. It may be possible to involve LLMs in designing algorithms for optimization problems. On one hand, we could study the theoretical guarantees of fully AI-generated algorithms. On the other hand, we could begin with a theoretically sound algorithm and use LLMs to help design its architecture. For existing algorithms, we may be able to use AI to reduce memory use or improve computational efficiency by reusing previous computations or using recursive definitions.

6 GenAI-use transparency

I used generative AI for brainstorming, spelling checks, and readability. The changes made by Copilot are summarized below:

- Spelling consistency

- Typographical corrections
- Grammar and punctuation
- Readability and sentence flow.

References

[Kha+24] Krishna Khadka et al. “A combinatorial approach to hyperparameter optimization”. In: *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*. 2024, pp. 140–149.

[Käs25] Christian Kästner. *Machine Learning in Production: From Models to Products*. Cambridge, MA: MIT Press, Apr. 2025.

[SB25] Camilo Chacón Sartori and Christian Blum. “Optimizing the Optimizer: An Example Showing the Power of LLM Code Generation”. In: *2025 20th Conference on Computer Science and Intelligence Systems (FedCSIS)*. IEEE. 2025, pp. 47–57.